

VERIQTA SOCIAL



VERIQTA

VERIQTA

VERIQTA

VERIQTA

VERIQTA

VERIQTA

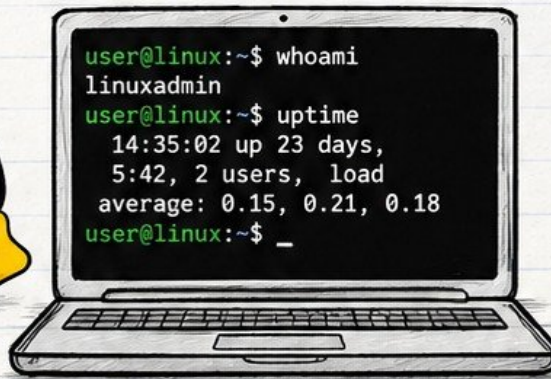
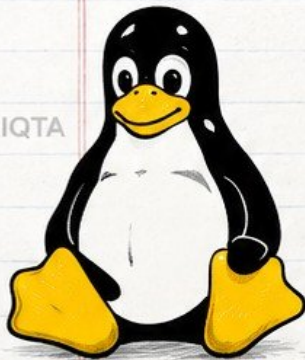
VERIQTA

VERIQTA

ULTIMATE LINUX

CHEAT SHEET

THE COMPLETE **20-PAGE** REFERENCE GUIDE
FOR EVERY LINUX ENGINEER



PRACTICAL
EXAMPLES



PRODUCTION
FOCUSED



SECURITY
BEST PRACTICES



TROUBLESHOOTING
WORKFLOWS



20

PAGES

CURATED CONTENT



500+

ESSENTIAL
COMMANDS



REAL WORLD
EXAMPLES

DEVOPS • SRE
SYSADMIN • CLOUD



CLOUD &
ON-PREMISE
READY



SECURE
PRODUCTION
ENVIRONMENTS



FROM BASICS TO PRODUCTION MASTERY
ONE CHEAT SHEET. ENDLESS SOLUTIONS.



- ✓ SAVE TIME
- ✓ WORK SMART
- ✓ SOLVE FASTER
- ✓ STAY SECURE

VERIQTA

VERIQTA SOCIAL



VERIQTA

LINUX

SYSTEM INFORMATION

QUICK REFERENCE FOR EVERY LINUX ENGINEER

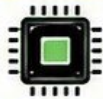
WHAT YOU CAN CHECK

OS AND DISTRIBUTION
INFORMATION

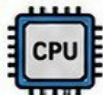
KERNEL VERSION



HOSTNAME



CPU ARCHITECTURE



CPU INFORMATION



MEMORY INFORMATION



HARDWARE INFORMATION



UPTIME



LOGGED-IN USERS

ESSENTIAL COMMANDS

COMMAND	PURPOSE	EXAMPLE OUTPUT (SAMPLE)												
<code>uname -a</code>	SHOW KERNEL AND SYSTEM INFORMATION	<pre>Linux server 5.15.0-100-generic #110-Ubuntu SMP x86_64 GNU/Linux</pre>												
<code>hostnamectl</code>	SHOW HOSTNAME AND OS DETAILS	<pre>Static hostname: server Operating System: Ubuntu 22.04.4 LTS Kernel: Linux 5.15.0-100-generic Architecture: x86-64</pre>												
<code>lscpu</code>	SHOW DETAILED CPU INFORMATION	<pre>Architecture: x86_64 CPU(s): 4 Model name: Intel(R) Xeon(R) CPU E5-2670 v3 @ 2.30GHz Thread(s) per core: 2 Core(s) per socket: 2</pre>												
<code>free -h</code>	SHOW MEMORY USAGE	<table><tr><th></th><th>total</th><th>used</th><th>free</th></tr><tr><td>Mem:</td><td>7.7Gi</td><td>2.1Gi</td><td>4.3Gi</td></tr><tr><td>Swap:</td><td>2.0Gi</td><td>0B</td><td>2.0Gi</td></tr></table>		total	used	free	Mem:	7.7Gi	2.1Gi	4.3Gi	Swap:	2.0Gi	0B	2.0Gi
	total	used	free											
Mem:	7.7Gi	2.1Gi	4.3Gi											
Swap:	2.0Gi	0B	2.0Gi											
<code>uptime</code>	SHOW SYSTEM UPTIME AND LOAD AVERAGE	<pre>14:32:10 up 23 days, 5:42, 2 users, load average: 0.15, 0.21, 0.18</pre>												
<code>who</code>	SHOW LOGGED-IN USERS	<pre>user1 pts/0 2024-05-15 14:21 (192.168.1.10) user2 pts/1 2024-05-15 14:25 (192.168.1.11)</pre>												
<code>w</code>	SHOW USERS AND SYSTEM ACTIVITY	<pre>14:32:10 up 23 days, 5:42, 2 users, load average: 0.15, 0.21, 0.18 USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT user1 pts/0 192.168.1.10 14:21 2:15m 0.41s 0.41s bash user2 pts/1 192.168.1.11 14:25 1:02m 0.31s 0.31s sshd</pre>												



QUICK TIP

These commands are your first step in understanding any Linux system. Use them before troubleshooting, deploying, or optimizing.

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

LINUX

FILESYSTEM NAVIGATION

NAVIGATE, EXPLORE AND MASTER THE LINUX FILESYSTEM

1. LINUX DIRECTORY STRUCTURE

 /	Root of the entire filesystem
 /etc	System configuration files
 /var	Variable data (logs, spool, cache)
 /home	User home directories
 /tmp	Temporary files (cleared on reboot)
 /usr	User programs and data
 /opt	Optional third-party software

2. NAVIGATE DIRECTORIES

-  **cd /path/to/directory**
Go to an absolute path
-  **cd ..**
Go up one directory
-  **cd ../../**
Go up multiple directories
-  **cd ~**
Go to your home directory
-  **cd -**
Go to previous directory

3. ABSOLUTE VS RELATIVE PATHS

ABSOLUTE PATH

`/home/user/documents/file.txt`

Starts from the root (/) and shows the full path.

RELATIVE PATH

`documents/file.txt`

Starts from the current directory.

4. LIST FILES

ls	List files in current directory
ls -l	Long format listing
ls -lh	Long format with human readable sizes
ls -a	Show all files (including hidden)
ls -la	Long format with hidden files
ls -R	Recursive listing (subdirectories)
ls -t	Sort by modification time
ls -S	Sort by file size

5. HIDDEN FILES

ls -a	Show all hidden files (start with .)
ls -la	Detailed list including hidden files
.*	Matches all hidden files and directories
~/.bashrc	Example of a hidden file

6. WORKING DIRECTORY



pwd
Print working directory
Shows your current location in the filesystem.

7. ESSENTIAL NAVIGATION COMMANDS

COMMAND	SYNTAX	DESCRIPTION	EXAMPLE	OUTPUT (EXAMPLE)
pwd	<code>pwd</code>	Print working directory	<code>pwd</code>	<code>/home/user</code>
cd	<code>cd /path</code>	Change directory	<code>cd /etc</code>	(no output)
ls	<code>ls [options]</code>	List files and directories	<code>ls -la</code>	total 24, drwxr-xr-x ...
tree	<code>tree [path]</code>	Display directory tree	<code>tree -L 2</code>	<pre>. ├── file1.txt ├── dir1 └── file2.txt</pre>

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

FILES AND DIRECTORIES

CREATE, MANAGE AND CONTROL YOUR FILES LIKE A PRO

1. CREATE FILES



touch filename

Create an empty file or update the timestamp of a file.

Example:

`touch notes.txt`

2. CREATE DIRECTORIES



mkdir directory

Create a new directory.

Example:

`mkdir projects`

3. COPY FILES OR DIRECTORIES



cp source dest

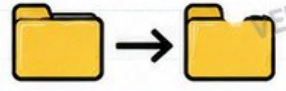
Copy files or directories to a new location.

Examples:

`cp file.txt /backup/`

`cp -r projects/ /backup/`

4. MOVE AND RENAME



mv source dest

Move or rename files or directories.

Examples:

`mv file.txt newname.txt`

`mv old_dir/ new_dir/`

5. DELETE FILES OR DIRECTORIES



rm target

Remove files or directories. Use with caution!

Examples:

`rm file.txt`

`rm -r old_dir/`

6. FILE TYPES (using file command)



Regular File

Text or binary file



Directory

Contains other files



Symbolic Link

Points to another file



Socket

Used for IPC communication



Named Pipe

Used for IPC communication



Device File

Hardware device access



Unknown

Unknown file type

file filename

Determine the type of a file.

Example:

`file /etc/passwd`

7. FILE METADATA (using stat command)



stat filename

Display detailed metadata about a file.

Example:

`stat notes.txt`

8. LINKS (using ln command)

HARD LINK



file.txt file_hard.txt

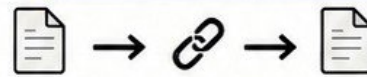
- Points to the same inode
- Cannot link across filesystems
- Deleting one does not affect the other

ln source hardlink

Example:

`ln file.txt file_hard.txt`

SYMBOLIC LINK (SOFT LINK)



file.txt symlink.txt file.txt

- Points to the path of the target file
- Can link across filesystems
- If target is deleted, link becomes broken

ln -s source symlink

Example:

`ln -s file.txt symlink.txt`

9. ESSENTIAL COMMAND REFERENCE

COMMAND	SYNTAX	DESCRIPTION	EXAMPLE	OUTPUT / RESULT
touch	<code>touch filename</code>	Create an empty file	<code>touch file.txt</code>	Creates file.txt
mkdir	<code>mkdir dirname</code>	Create a new directory	<code>mkdir new_dir</code>	Creates new_dir/
cp	<code>cp source dest</code>	Copy files or directories	<code>cp file.txt /backup/</code>	Copies file.txt to /backup/
mv	<code>mv source dest</code>	Move or rename files/directories	<code>mv old.txt new.txt</code>	Renames old.txt to new.txt
rm	<code>rm target</code>	Remove files or directories	<code>rm file.txt</code>	Deletes file.txt
file	<code>file filename</code>	Determine file type	<code>file /bin/bash</code>	/bin/bash: symbolic link
stat	<code>stat filename</code>	Show file metadata	<code>stat file.txt</code>	Displays detailed info
ln	<code>ln [options] target linkname</code>	Create links (hard or symbolic)	<code>ln -s file.txt link.txt</code>	Creates symbolic link

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VIEWING AND SEARCHING FILE CONTENT

READ, SEARCH AND EXPLORE FILES LIKE A PRO

1. READ FILES



cat filename
Display entire file content.
Example:
`cat notes.txt`

less filename
View file page by page.
Use: SPACE (next page),
b (back), q (quit).
Example:
`less /var/log/syslog`

2. FIRST AND LAST LINES



head filename
Show the first 10 lines (default).
Example:
`head file.txt`
`head -n 20 file.txt`
Show first 20 lines.

tail filename
Show the last 10 lines (default).
Example:
`tail file.txt`
`tail -n 20 file.txt`
Show last 20 lines.

3. FOLLOW CHANGING FILES



tail -f filename
Follow file content in real time.
Example:
`tail -f /var/log/syslog`



GREAT FOR LOGS

See new lines as they are written to the file.

4. SEARCH TEXT



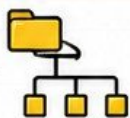
grep "text" filename
Search for "text" in a file.
Example:
`grep "error" app.log`

grep -i "text" file
Case-insensitive search.

grep -n "text" file
Show line numbers.

grep -r "text" dir/
Search recursively in a directory.

5. RECURSIVE SEARCH



grep -r "text" dir/
Search for "text" in all files under a directory.

Example:
`grep -r "TODO" /home/user`

6. FIND FILES



find /path -name "file.txt"
Find files by name.
Example:
`find /home -name "*.log"`
find /path -type d -name "dir"
Find directories by name.
Example:
`find / -type d -name "config"`

find /path -size +100M
Find files larger than 100MB.

7. LOCATE COMMANDS



locate filename
Quickly find files using updated database.
Example:
`locate sshd_config`



UPDATE DATABASE

`sudo updatedb`
Update the locate database (manually or via cron).

8. WHICH COMMANDS



which command
Show full path of a command.

Example:
`which ls`

Output:
`/bin/ls`

9. WHEREIS COMMANDS



whereis command
Show binary, source and man page locations.

Example:
`whereis python`
Output:
`python: /usr/bin/python3
/usr/lib/python3.11 /usr/share/man/man1/python.1.gz`

10. QUICK COMMAND REFERENCE

COMMAND	SYNTAX	DESCRIPTION	EXAMPLE	OUTPUT / RESULT
cat	cat filename	Display entire file	cat file.txt	Shows full content
less	less filename	View file interactively	less file.txt	Page-by-page view
head	head -n N file	Show first N lines	head -n 20 file.txt	First 20 lines
tail	tail -n N file	Show last N lines	tail -n 20 file.txt	Last 20 lines
tail -f	tail -f file	Follow file in real time	tail -f app.log	Live new lines
grep	grep "text" file	Search text in file	grep "error" log.txt	Matching lines
grep -r	grep -r "text" dir	Recursive search	grep -r "TODO" src/	Matches in all files
find	find /path -name "*"	Find files by criteria	find /var -name "*.log"	Matching files
locate	locate filename	Find files using database	locate hosts	Fast file list
which	which command	Show command path	which python	/usr/bin/python
whereis	whereis command	Show binary, src, man	whereis git	Binary, source, man paths



QUICK TIP

Use `less` for large files, `tail -f` for logs, `grep -r` for code search, `find` for precise file discovery, and `locate` for speed!

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

TEXT PROCESSING POWER TOOLS

— FILTER, TRANSFORM AND ANALYZE TEXT LIKE A PRO —

1. SEARCH AND FILTER



```
grep 'pattern' file
```

Search for text patterns.

Examples:

```
grep 'error' app.log
grep -i 'test' file.txt
grep -r 'TODO' src/
grep -v 'debug' log.txt
grep -n 'main' file.c
```

2. SORT



```
sort [options] file
```

Sort lines of text.

Examples:

```
sort file.txt
sort -r file.txt
sort -n numbers.txt
sort -k 2 data.txt
sort -u file.txt
```

3. REMOVE DUPLICATES



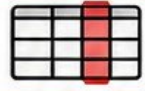
```
uniq [options] file
```

Remove or display duplicate lines.

Examples:

```
sort file.txt | uniq
sort file.txt | uniq -d
sort file.txt | uniq -u
uniq -c file.txt
```

4. EXTRACT COLUMNS



```
cut [options] file
```

Extract sections (columns) from each line.

Examples:

```
cut -d',' -f1 file.csv
cut -d':' -f1,3 /etc/passwd
cut -c1-10 file.txt
cut -f2-4 -d' ' file.txt
```

5. TRANSFORM TEXT



```
tr 'a-z' 'A-Z'
```

Translate or delete characters.

Examples:

```
tr 'a-z' 'A-Z' < file.txt
tr -d '0-9' < file.txt
tr ' ' '_' < file.txt
tr -s ' ' < file.txt
```

6. COUNT OUTPUT



```
wc [options] file
```

Count lines, words, and characters.

Examples:

```
wc file.txt
wc -l file.txt
wc -w file.txt
wc -c file.txt
```

7. PIPELINES



```
command1 | command2
```

Pass output of one command as input to another.

Examples:

```
cat file.txt | grep 'error'
ps aux | sort -k 3 -nr
df -h | grep '^/dev/'
cat file.txt | wc -l
```

8. FIELD PROCESSING



```
awk '{print $1}' file
```

Pattern scanning and processing language.

Examples:

```
awk '{print $1}' file.txt
awk -F: '{print $1}' /etc/passwd
awk 'sum+= $1 END {print sum}'
awk '$3 > 100 {print $0}' file.txt
```

9. STREAM EDITOR



```
sed 's/old/new/g' file
```

Stream editor for text transformations.

Examples:

```
sed 's/error/ERROR/g' file.txt
sed -i 's/test/TEST/g' file.txt
sed '1,5d' file.txt
sed 's/^/# /' file.txt
```

10. EXECUTE COMMANDS FROM INPUT



```
xargs [command]
```

Build and execute command lines from standard input.

Examples:

```
find . -name '*.log' | xargs grep 'error'
cat files.txt | xargs rm -f
echo 'one two three' | xargs -n 1 echo
```

11. COMBINE MULTIPLE TOOLS



```
cat file.txt | grep 'error' |
sort | uniq -c | sort -nr
```

Powerful combinations for real-world text processing.

Examples:

```
ps aux | awk '{print $11}' | sort | uniq -c
grep -r 'TODO' . | cut -d: -f1 | sort | uniq
cat access.log | awk '{print $1}' | sort | uniq -c
```

12. QUICK COMMAND REFERENCE TABLE

COMMAND	PURPOSE	SYNTAX EXAMPLE	EXAMPLE USE CASE	SAMPLE OUTPUT
grep	Search text	grep 'pattern' file	Find errors in logs	error: failed to connect
awk	Process fields	awk '{print \$1}' file	Extract first column	user1
sed	Edit/replace text	sed 's/old/new/g' file	Replace old with new	new value here
cut	Extract columns	cut -d' ' -f1 file	Get first field in CSV	id,name,age → id
sort	Sort lines	sort file	Sort names or data	a b c
uniq	Remove duplicates	sort file uniq	List unique items	unique lines only
tr	Translate/delete	tr 'a-z' 'A-Z'	Uppercase text	HELLO WORLD
wc	Count output	wc -l file	Count lines	20
xargs	Run commands	echo file xargs rm	Delete listed files	(files deleted)

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

USERS AND GROUPS

MANAGE USERS, GROUPS AND ACCESS LIKE A PRO

1. CURRENT USER

**whoami**

Display the current logged-in username.

Example:

`whoami`

Output:

`john`

2. USER INFORMATION

**id [username]**

Display user and group IDs.

Example:

`id`

Output:

`uid=1000(john) gid=1000(john)
groups=1000(john),10(wheel)`

`id username`

Show info for a specific user.

`id alice`

3. CREATE USERS

**useradd [options] username**

Create a new user.

Example:

`useradd alice`

Create user with home directory:

`useradd -m alice`

Create with custom shell:

`useradd -m -s /bin/bash alice`

4. DELETE USERS

**userdel [options] username**

Delete a user.

Example:

`userdel alice`

Delete and remove home directory:

`userdel -r alice`

5. MODIFY USERS

**usermod [options] username**

Common options:

- `-aG group` Add user to a group
- `-g group` Change primary group
- `-s shell` Change login shell
- `-l newname` Rename user
- `-L` Lock user account
- `-U` Unlock user account

Examples:

`usermod -aG sudo alice`
`usermod -s /bin/zsh alice`
`usermod -l newalice alice`

6. GROUPS

**groupadd [groupname]**

Create a new group.

Example:

`groupadd devops`

View groups for current user:

`groups`

View groups for a user:

`groups alice`

7. PASSWORDS

**passwd [username]**

Change user password.

Example:

`passwd`

Change another user password (root required):

`passwd alice`

Force password change at next login:

`passwd -e alice`

8. SWITCH USERS

**su [username]**

Switch to another user.

Example:

`su alice`

Switch to root:

`su -`
(or `su - root`)

Exit with: `exit`

9. ESSENTIAL USER & GROUP COMMAND REFERENCE

COMMAND	SYNTAX	DESCRIPTION	EXAMPLE	OUTPUT / RESULT
<code>whoami</code>	<code>whoami</code>	Show current username	<code>whoami</code>	<code>john</code>
<code>id</code>	<code>id [username]</code>	Show user and group IDs	<code>id alice</code>	<code>uid=1001(alice) gid=1001(alice) ...</code>
<code>useradd</code>	<code>useradd [options] username</code>	Create a new user	<code>useradd -m alice</code>	User 'alice' created
<code>userdel</code>	<code>userdel [options] username</code>	Delete an existing user	<code>userdel -r alice</code>	User and home removed
<code>usermod</code>	<code>usermod [options] username</code>	Modify user account	<code>usermod -aG sudo alice</code>	User added to sudo group
<code>groupadd</code>	<code>groupadd groupname</code>	Create a new group	<code>groupadd devops</code>	Group 'devops' created
<code>passwd</code>	<code>passwd [username]</code>	Set or change password	<code>passwd alice</code>	Password updated
<code>su</code>	<code>su [username]</code>	Switch user	<code>su alice</code>	Switched to alice



QUICK TIPS

- ✓ Use strong passwords and change regularly.
- ✓ Use groups to manage permissions efficiently.
- ✓ Always use least privilege and avoid unnecessary root access.



SECURITY REMINDER

Review `/etc/passwd` and `/etc/shadow` only as root. Protect these files.

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

LINUX PERMISSIONS AND OWNERSHIP

CONTROL ACCESS. PROTECT YOUR SYSTEM.

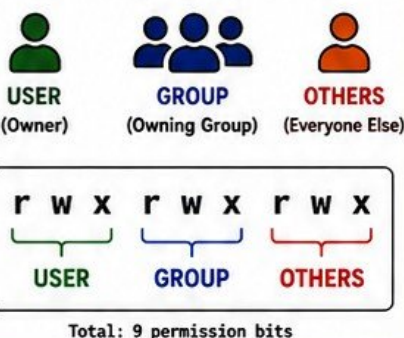
1. PERMISSION BASICS

Every file and directory has 3 types of permissions:

r READ	4	View file contents List directory contents
w WRITE	2	Modify file contents Create/delete files in directory
x EXECUTE	1	Run file as program/script Enter directory

2. PERMISSION MODEL

Each file has 3 permission sets:



3. NUMERIC PERMISSIONS

Permissions are represented using numbers (octal):

PERM	BINARY	OCTAL	MEANING
rwX	111	7	Read, Write, Execute
rw-	110	6	Read, Write
r-X	101	5	Read, Execute
r--	100	4	Read only
-wX	011	3	Write, Execute
-w-	010	2	Write only
--X	001	1	Execute only
---	000	0	No permissions

4. SYMBOLIC PERMISSIONS

Use letters to add (+), remove (-) or set (=) permissions.

WHO	EXAMPLES
u User (owner)	u+x Add execute permission for user
g Group	g-w Remove write permission for group
o Others	o=r Set others to read only
a All (u,g,o)	a=rwx Give all full permissions
	u=rw,g=r,o=r Set exact permissions

5. CHANGE PERMISSIONS (chmod)

CHANGE PERMISSIONS WITH chmod

SYNTAX:

```
chmod [options] mode file
```

EXAMPLES:

```
chmod 644 file.txt      # -rw-r--r--
chmod 755 script.sh     # -rwxr-xr-x
chmod 600 secret.txt    # -rw-----
chmod -R 755 /var/www   # Recursive
```

6. CHANGE OWNERSHIP

CHANGE OWNER (chown)

Syntax: `chown user file`

Example: `chown alice file.txt`

CHANGE GROUP (chgrp)

Syntax: `chgrp group file`

Example: `chgrp devops file.txt`

CHANGE BOTH OWNER & GROUP

Syntax: `chown user:group file`

Example: `chown alice:devops file.txt`

7. RECURSIVE OPERATIONS

Use **-R** to apply changes recursively to directories and their contents.

EXAMPLES:

```
chmod -R 755 /opt/app
chown -R alice:devops /opt/app
chgrp -R devops /opt/app
```



BE CAREFUL!

Recursive changes affect all files and subdirectories. Always double-check paths.

8. COMMON PERMISSION SETS

PERMISSIONS	NUMERIC	DESCRIPTION	COMMON USE
644	-rw-r--r--	Read/Write for owner, Read for group & others	Files
755	-rwxr-xr-x	Full access for owner, Read/Execute for group & others	Directories, Scripts
600	-rw-----	Read/Write for owner only	Private files, Configs
700	-rwx-----	Full access for owner only	Private directories, Scripts

9. PERMISSION TROUBLESHOOTING

PROBLEM	CHECK & SOLUTION
Permission denied	<code>ls -l file</code> Check permissions and ownership
Cannot execute script	<code>chmod +x script.sh</code> Ensure executable bit is set
Cannot delete file	Check write permission on directory
Cannot access directory	Check "x" permission on directory
Wrong owner/group	Use <code>chown</code> or <code>chgrp</code> to fix ownership

10. QUICK PERMISSION REFERENCE

```
ls -l file.txt
-rw-r--r-- 1 alice devops 1234 May 10 10:00 file.txt
```

FILE TYPE PERMISSIONS (rwx bits) LINKS (Hard links) OWNER (User) GROUP (Group) SIZE (Bytes) DATE & TIME NAME (Filename)

FILE TYPES

- Regular file
- d Directory
- l Symbolic link
- b Block device
- c Character device
- p Named pipe
- s Socket

TIPS

- ✓ Least privilege is best practice.
- ✓ Use 644 for files, 755 for directories.
- ✓ Never use 777 in production.
- ✓ Always backup before recursive changes.

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

ADVANCED PERMISSIONS AND PRIVILEGE

CONTROL PRIVILEGES. PROTECT YOUR SYSTEM.

1. SUDO – RUN COMMANDS AS ANOTHER USER

\$ sudo command

- Run commands with elevated privileges.
- By default, uses your own password.
- Add sudo to the beginning of a command.

EXAMPLES:

```
sudo apt update
sudo systemctl restart nginx
sudo -i # Root shell
sudo -u alice whoami # Run as user
```

2. /ETC/SUDOERS – SUDO CONFIGURATION



- File that defines who can run what as root.
- Location: /etc/sudoers
- Edit ONLY with visudo.



WARNING:

Never edit /etc/sudoers directly.
Always use visudo.

3. VISUDO – EDIT SUDOERS SAFELY

\$ visudo

- Opens /etc/sudoers in a safe editor.
- Validates syntax before saving.
- Prevents lockout from syntax errors.

EXAMPLE (Grant sudo access):

```
alice ALL=(ALL:ALL) ALL
```

alice can run all commands as all users on all hosts.

4. SUID – SET USER ID



SUID

- Allows users to run a file with the permissions of the file owner.
- Shown as 's' in user execute bit.

EXAMPLE:

passwd command has SUID so normal users can change passwords.

```
# Set SUID
chmod u+s /path/to/file
```

Permissions Example:

```
-rwsr-xr-x (4755)
```

5. SGID – SET GROUP ID



SGID

- Allows users to run a file with the permissions of the file group.
- Shown as 's' in group execute bit.
- On directories: new files inherit the group.

EXAMPLE:

Shared directories maintain group ownership.

```
# Set SGID
chmod g+s /path/to/dir
```

Permissions Example:

```
drwxr-sr-x (2755)
```

6. STICKY BIT



STICKY

- On directories: only file owner, directory owner, or root can delete files.
- Prevents accidental deletion of others' files.
- Shown as 't' in others execute bit.

EXAMPLE:

/tmp has sticky bit set.

```
# Set Sticky Bit
chmod +t /path/to/dir
```

Permissions Example:

```
drwxrwxrwt (1777)
```

7. UMASK – DEFAULT PERMISSIONS



UMASK

- Controls the default permissions for new files and directories.
- Subtracts permissions from the maximum allowed.
- Default for files: 666
- Default for directories: 777

EXAMPLE:

```
umask 022
```

New files: 644
New directories: 755

Check current umask:

```
umask
```

8. ACLs – ACCESS CONTROL LISTS



- Provide fine-grained permissions beyond traditional owner/group/others.
- Useful for shared environments.
- Require filesystem support (ext4, xfs, etc).

ACL PERMISSION TYPES

r	Read
w	Write
x	Execute
-	None

COMMON ACL COMMANDS

getfacl file	View ACLs
setfacl -m u:user:perms file	Modify ACL
setfacl -x u:user file	Remove ACL
setfacl -b file	Remove all ACLs
setfacl -d -m g:group:perms dir	Set default ACL for dir

9. GETFACL – VIEW ACLs

```
$ getfacl file_or_directory
```

EXAMPLE OUTPUT:

```
# file: shared.txt
# owner: alice
# group: devops
user::rw-
user:bob:r--
group::r-x
mask::r--
other::r--
```

10. SETFACL – SET ACLs

```
$ setfacl [options] file
```

EXAMPLES:

```
# Give user bob read access
setfacl -m u:bob:r-- file.txt

# Give group devs read/write
setfacl -m g:devs:rw- file.txt

# Default ACL for new files in dir
setfacl -d -m g:devs:rw- /project
```

11. SAFE PRIVILEGE MANAGEMENT BEST PRACTICES



- ✓ FOLLOW THE PRINCIPLE OF LEAST PRIVILEGE
- ✓ USE SUDO INSTEAD OF ROOT LOGIN
- ✓ LIMIT SUDO ACCESS TO REQUIRED USERS ONLY
- ✓ LOG AND MONITOR SUDO USAGE
- ✓ USE STRONG PASSWORDS AND KEY-BASED AUTHENTICATION

- ✓ REGULARLY REVIEW /ETC/SUDOERS AND ACLs
- ✓ AVOID SUID/SGID ON CUSTOM SCRIPTS
- ✓ REMOVE UNUSED ACCOUNTS AND GROUPS
- ✓ KEEP SYSTEM AND PACKAGES UP TO DATE
- ✓ BACKUP CRITICAL CONFIGURATION FILES

QUICK REFERENCE SUMMARY

sudo	Run command as another user
visudo	Edit sudoers safely
chmod u+s	Set SUID
chmod g+s	Set SGID
chmod +t	Set Sticky Bit
umask	Set default permissions
getfacl	View ACLs
setfacl	Modify ACLs



SECURITY IS EVERYONE'S RESPONSIBILITY.
USE POWER CAREFULLY. USE ACCESS WISELY.

VERIQTA SOCIAL



PROCESSES AND JOB CONTROL

MANAGE PROCESSES. CONTROL JOBS. RUN LIKE A PRO.

1. LIST PROCESSES

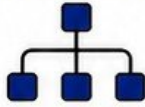
**ps [options]**

- Show a snapshot of current processes.
- Essential for troubleshooting and monitoring.

EXAMPLES:

```
ps aux
ps -ef
ps -u $USER
ps -o pid,ppid,cmd,%mem,%cpu
```

2. PROCESS HIERARCHY

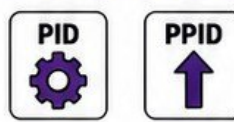


- Every process has a Parent (PPID).
- Forms a tree-like structure.
- Init/systemd (PID 1) is the root.

EXAMPLE TREE:

PID	PPID	COMMAND
1	0	systemd
123	1	sshd
456	123	bash
789	456	vim

3. PID AND PPID



- PID: Process ID (unique number).
- PPID: Parent Process ID.
- Useful for tracing processes.

EXAMPLE:

```
ps -o pid,ppid,cmd -p 456
```

PID	PPID	CMD
456	123	bash

4. FOREGROUND & BACKGROUND JOBS



- Foreground: Runs in current terminal.
- Background: Runs in background.
- Use job control to manage.

EXAMPLES:

```
command &          # Run in background
Ctrl + Z           # Pause to background
jobs                # List background jobs
fg %1               # Bring job 1 to foreground
bg %1               # Send job 1 to background
```

5. SIGNALS



- Signals are used to control processes.
- Each signal has a number and purpose.

COMMON SIGNALS:

SIGNAL	NO.	PURPOSE
SIGHUP	1	Terminal hangup
SIGINT	2	Interrupt (Ctrl+C)
SIGTERM	15	Terminate process
SIGKILL	9	Kill process (force)
SIGSTOP	19	Stop (pause) process
SIGCONT	18	Continue stopped process

6. KILL PROCESSES



- Terminate processes using signals.
- Use carefully!

EXAMPLES:

```
kill 1234          # Send SIGTERM
kill -9 1234       # Send SIGKILL
pkill nginx        # Kill by name
pkill -9 java      # Force kill by name
```

7. PRIORITY AND NICE



- Lower nice value = higher priority.
- Higher nice value = lower priority.
- Default nice value is 0.

EXAMPLES:

```
nice -n 10 command # Start with nice
renice -n 5 -p 1234 # Change nice value
top                 # View processes by priority
ps -o pid,ni,cmd -p 1234
                    # Show nice value (NI column)
```

8. USEFUL PROCESS FILTERING



- Filter and find processes easily.
- Save time with smart commands.

EXAMPLES:

```
pgrep nginx        # Find PID(s) by name
pgrep -u $USER      # Find by user
pgrep -f 'python'   # Find by full command
ps aux | grep nginx # Search in listing
ps -u $USER -o pid,cmd,%cpu |
sort -k3 -nr | head # Top CPU by user
```

9. ESSENTIAL PROCESS & JOB CONTROL COMMAND REFERENCE

COMMAND	SYNTAX	DESCRIPTION	EXAMPLE	OUTPUT / RESULT
ps	ps aux	Display running processes	ps aux	List of all running processes
top	top	Real-time process monitor	top	Interactive process dashboard
pgrep	pgrep [name]	Find PID by process name	pgrep nginx	1234
jobs	jobs	List background jobs	jobs	[1]+ Running command &
bg	bg %1	Resume job in background	bg %1	[1]+ command &
fg	fg %1	Bring job to foreground	fg %1	command
kill	kill [signal] PID	Send signal to process	kill -9 1234	Process terminated
pkill	pkill [name]	Kill by process name	pkill nginx	All nginx processes killed
nice	nice -n N command	Start command with priority	nice -n 10 python app.py	Run with lower priority
renice	renice -n N -p PID	Change priority of running process	renice -n 5 -p 1234	Priority updated
ps -o	ps -o pid,ppid,cmd -p PID	Custom format process info	ps -o pid,ppid,cmd -p 456	456 123 bash



QUICK TIPS

- ✓ Use ps aux to quickly see what's running.
- ✓ Use pgrep to find PIDs by name.
- ✓ Use kill -15 first (SIGTERM), then kill -9 if needed.
- ✓ Never kill system or critical processes (PID 1, systemd).



SAFETY REMINDER

Use kill and kill -9 carefully.
When in doubt, verify before terminating any process.

COMMON SIGNAL SHORTCUTS

```
Ctrl + C  → SIGINT (2)
Ctrl + Z  → SIGTSTP (20)
kill PID  → SIGTERM (15)
kill -9 PID → SIGKILL (9)
```

VERIQTА SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

SYSTEMD AND SERVICE MANAGEMENT

MANAGE SERVICES. AUTOMATE RELIABLY.

1. SERVICE STATUS



- Check if a service is running.
- View detailed status and logs.

COMMAND:

```
systemctl status <service>
```

EXAMPLE:

```
systemctl status nginx
```

2. START / STOP SERVICES



- Start a stopped service.
- Stop a running service.

COMMANDS:

```
systemctl start <service>  
systemctl stop <service>
```

EXAMPLE:

```
systemctl start nginx  
systemctl stop nginx
```

3. RESTART vs RELOAD



RESTART

RELOAD

- Restart: Stop and start again.
- Reload: Apply changes without stopping (if supported).

COMMANDS:

```
systemctl restart <service>  
systemctl reload <service>
```

EXAMPLE:

```
systemctl restart nginx  
systemctl reload nginx
```

4. ENABLE / DISABLE AT BOOT



ENABLE

DISABLE

- Enable: Start at boot.
- Disable: Do not start at boot.

COMMANDS:

```
systemctl enable <service>  
systemctl disable <service>
```

EXAMPLE:

```
systemctl enable nginx  
systemctl disable nginx
```

5. FAILED SERVICES



- List services that failed to start.
- Quickly identify issues.

COMMAND:

```
systemctl --failed
```

FIX EXAMPLE:

1. systemctl --failed
2. systemctl status <service>
3. Check logs and fix issue
4. systemctl restart <service>

6. UNIT FILES



- Unit files define how services run.
- Located in /etc/systemd/system/ or /lib/systemd/system/

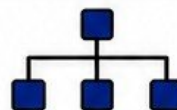
VIEW UNIT FILE:

```
systemctl cat <service>
```

EDIT / CREATE UNIT FILES:

```
sudo systemctl edit <service>
```

7. DEPENDENCIES



- Services can depend on other services or targets.
- Ensure correct start order.

COMMON COMMANDS:

```
systemctl list-dependencies <service>
```

```
systemctl list-dependents <service>
```

```
systemctl show <service> | grep -i  
"Requires\|Wants\|After\|Before"
```

8. ESSENTIAL SYSTEMCTL COMMANDS



VIEW ALL ACTIVE UNITS:

```
systemctl list-units --type=service
```

VIEW ALL UNITS (ALL STATES):

```
systemctl list-units --all
```

VIEW UNIT DETAILS:

```
systemctl show <service>
```

VIEW UNIT FILE PATH:

```
systemctl status <service> | grep Loaded
```

9. SYSTEMCTL COMMAND REFERENCE

COMMAND	DESCRIPTION	EXAMPLE	OUTPUT / PURPOSE
systemctl status <service>	Show status of a service	systemctl status sshd	Active: active (running)
systemctl start <service>	Start a service	systemctl start httpd	Starts the service
systemctl stop <service>	Stop a service	systemctl stop httpd	Stops the service
systemctl restart <service>	Restart a service	systemctl restart nginx	Stops then starts again
systemctl reload <service>	Reload service configuration	systemctl reload nginx	Reloads config without stop
systemctl enable <service>	Enable service at boot	systemctl enable nginx	Creates symlink for boot
systemctl disable <service>	Disable service at boot	systemctl disable nginx	Removes boot symlink
systemctl --failed	List all failed units	systemctl --failed	Shows failed services
systemctl list-units --type=service	List active services	systemctl list-units --type=service	List of running services
systemctl list-units --all	List all units (all states)	systemctl list-units --all	All loaded units
systemctl cat <service>	Show unit file contents	systemctl cat sshd	Displays unit configuration
journalctl -u <service>	View logs for a service	journalctl -u nginx -f	Real-time logs

BEST PRACTICES



- ✓ Use enable to ensure critical services start at boot.
- ✓ Prefer reload over restart when supported.
- ✓ Regularly check failed services.
- ✓ Always review logs for troubleshooting.

TROUBLESHOOTING QUICK CHECK



1. systemctl status <service>
2. journalctl -u <service> --no-pager
3. Check unit file: systemctl cat <service>
4. Verify dependencies
5. Restart or reload

COMMON LOCATIONS



Unit files (system): /lib/systemd/system/
Unit files (custom): /etc/systemd/system/
Drop-in overrides:
/etc/systemd/system/<service>.d/
Logs: journalctl -u <service>

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

11

PACKAGE MANAGEMENT

Page 11 of 20

apt • dpkg • dnf • yum • rpm COMMANDS CHEAT SHEET



PACKAGE MANAGERS HELP YOU INSTALL, REMOVE, UPDATE, SEARCH, AND MANAGE SOFTWARE ON LINUX SYSTEMS.

DEBIAN / UBUNTU (APT & DPKG)



UPDATE REPOSITORIES

```
sudo apt update
```



UPGRADE PACKAGES

```
sudo apt upgrade -y
```



INSTALL A PACKAGE

```
sudo apt install <package>
```



REMOVE A PACKAGE

```
sudo apt remove <package>
```



SEARCH FOR PACKAGES

```
apt search <keyword>
```



PACKAGE INFORMATION

```
apt show <package>
```



LIST INSTALLED PACKAGES

```
dpkg -l
```



CHECK PACKAGE FILES

```
dpkg -L <package>
```



INSTALL A .DEB FILE

```
sudo dpkg -i <file.deb>
```



FIX BROKEN DEPENDENCIES

```
sudo apt --fix-broken install
```

RHEL / ROCKY / ALMALINUX (DNF / YUM & RPM)



UPDATE REPOSITORIES

```
sudo dnf makecache  
(or sudo yum makecache)
```



UPGRADE PACKAGES

```
sudo dnf upgrade -y  
(or sudo yum update -y)
```



INSTALL A PACKAGE

```
sudo dnf install <package>  
(or sudo yum install <package>)
```



REMOVE A PACKAGE

```
sudo dnf remove <package>  
(or sudo yum remove <package>)
```



SEARCH FOR PACKAGES

```
dnf search <keyword>  
(or yum search <keyword>)
```



PACKAGE INFORMATION

```
dnf info <package>  
(or yum info <package>)
```



LIST INSTALLED PACKAGES

```
dnf list installed  
(or yum list installed)
```



REPOSITORY INFORMATION

```
dnf repolist all  
(or yum repolist all)
```



INSTALL AN .RPM FILE

```
sudo rpm -ivh <file.rpm>
```



VERIFY AN .RPM FILE

```
rpm -K <file.rpm>
```

RPM COMMANDS (LOW LEVEL)



INSTALL AN RPM

```
sudo rpm -ivh <file.rpm>
```



REMOVE AN RPM

```
sudo rpm -e <package>
```



QUERY AN RPM

```
rpm -q <package>
```



LIST FILES IN RPM

```
rpm -ql <package>
```



RPM INFORMATION

```
rpm -qi <package>
```



VERIFY RPM

```
rpm -V <package>
```



QUICK COMMANDS SUMMARY

- UPDATE REPOS → apt update / dnf makecache
- UPGRADE SYSTEM → apt upgrade -y / dnf upgrade -y
- INSTALL PACKAGE → apt install pkg / dnf install pkg
- REMOVE PACKAGE → apt remove pkg / dnf remove pkg
- SEARCH PACKAGE → apt search kw / dnf search kw
- PACKAGE INFO → apt show pkg / dnf info pkg



TIPS

- Use apt on Debian, Ubuntu, Linux Mint, etc.
- Use dnf (or yum) on RHEL, Rocky, AlmaLinux, CentOS Stream, Fedora.
- Always update repositories before installing.
- Prefer dnf over yum on modern RHEL-based systems.
- Use rpm for manual RPM file management.

VERIQTA SOCIAL



12

DISK AND FILESYSTEM
MANAGEMENT

df • du • lsblk • blkid • mount • umount • findmnt

MONITOR DISK USAGE, MANAGE FILESYSTEMS,
AND DISCOVER STORAGE ON LINUX SYSTEMS.1. DISK USAGE
OVERALL SUMMARY**df -h**Show filesystem disk
usage in human
readable format.2. FILESYSTEM
CAPACITY**df -Th**Show disk usage
with filesystem types.3. BLOCK DEVICES
OVERVIEW**lsblk**List all block devices
in a tree format.4. BLOCK DEVICE
DETAILS**lsblk -f**List block devices
with filesystem
information.5. FILESYSTEM UUID
AND LABEL**blkid**Show UUIDs, LABELs
and filesystem types
of block devices.6. MOUNT POINTS
IN USE**mount**Show currently
mounted filesystems.7. BETTER MOUNT
OVERVIEW**findmnt**Modern way to show
mounts in a tree view.

8. MOUNT A FILESYSTEM

**mount /dev/sdX1 /mnt/data**Mount a device to
a directory.9. UNMOUNT A
FILESYSTEM**umount /mnt/data**
Unmount a filesystem
from a directory.10. UNMOUNT
BY DEVICE**umount /dev/sdX1**
Unmount a device
directly.

11. INODE USAGE

**df -i**Show inode usage
for filesystems.12. FIND LARGE
DIRECTORIES**du -sh /***Find size of directories
in root (summary).

13. FIND LARGE FILES

find / -type f -size +100M -exec du -h {} \; 2>/dev/null
Find files larger than 100MB (change size as needed).

TIP

Use **du -sh *** inside a directory
to find which folders consume
the most space.

COMMON COMMAND EXAMPLES

df -h	Show disk usage in human readable format.
df -Th	Show disk usage with filesystem types.
lsblk	List all block devices in tree format.
lsblk -f	List block devices with filesystem info.
blkid	Show UUIDs, LABELs and filesystem types.
mount	Show mounted filesystems.
findmnt	Show mounted filesystems in tree view.
mount /dev/sdX1 /mnt/data	Mount a device.
umount /mnt/data	Unmount a filesystem.
df -i	Show inode usage.
du -sh /path	Show size of a directory.
find / -type f -size +100M -exec du -h {} \;	Find large files.

QUICK REFERENCE TABLE

COMMAND	PURPOSE
df	Filesystem disk space usage
du	Directory and file disk usage
lsblk	Block device listing
blkid	Device UUID and filesystem info
mount	Show mounted filesystems
umount	Unmount filesystems
findmnt	Show mounts in tree view



IMPORTANT NOTES

- Always check available space before installations.
- Inodes matter: full inodes = can't create new files.
- Use **findmnt** instead of **mount** for better output.
- Be careful when mounting/unmounting in production.
- Use **umount -l** only as a last resort.

13

MEMORY, CPU, AND PERFORMANCE

Page 13 of 20

free • vmstat • iostat • mpstat • top • uptime

**MONITOR SYSTEM RESOURCES, IDENTIFY PERFORMANCE BOTTLENECKS, AND DIAGNOSE ISSUES IN REAL TIME.**

1. MEMORY USAGE

**free -h**

Show total, used, free memory and buffers/cache.

	total	used	free	buff/cache	available
Mem:	15Gi	6.1Gi	2.0Gi	6.9Gi	8.2Gi
Swap:	2.0Gi	256Mi	1.8Gi		

2. SWAP USAGE

**free -h**

Check swap space usage and availability.

	total	used	free	%used
Swap:	2.0Gi	256Mi	1.8Gi	12%

3. CPU UTILIZATION

**top**

Real-time CPU usage per process.

```
top - 14:35:02 up 23 days,  2:10,  2 users,
load average: 0.15, 0.21, 0.18
%Cpu(s):  2.1 us,  1.0 sy,  0.0 ni, 96.5 id,  0.4 wa,
            0.0 hi,  0.0 si,  0.0 st
```

PID	USER	%CPU	%MEM	COMMAND
1234	root	2.1	1.2	systemd
2345	nginx	1.5	2.5	nginx
3456	root	0.9	0.7	sshd

4. LOAD AVERAGE

**uptime**

Show system load averages (1, 5, 15 minutes).

```
14:35:02 up 23 days,  2:10,  2 users,
load average: 0.15, 0.21, 0.18
```

5. I/O PRESSURE & DISK STATS

**iostat -xz 1 5**

Monitor disk I/O, utilization, and performance.

Device	r/s	w/s	rkB/s	wkB/s	avgrr-sz	%util
sda	1.2	0.8	24.1	12.3	21.9	2.1
sdb	0.0	1.1	0.0	18.4	33.5	1.1
nvme0n1	2.5	3.7	120.5	98.2	35.6	7.2

6. CPU PERFORMANCE (PER CORE)

**mpstat -P ALL 1 5**

Show CPU usage for each core.

CPU	%usr	%nice	%sys	%iowait	%idle
all	1.20	0.00	0.80	0.40	97.60
0	1.10	0.00	0.90	0.30	97.70
1	1.30	0.00	0.70	0.40	97.60
2	1.20	0.00	0.80	0.40	97.60
3	1.20	0.00	0.80	0.50	97.50



CPU vs MEMORY vs DISK DIAGNOSIS



CPU BOUND

- High %usr or %sys in top/mpstat
- Load average high, CPU idle low
- Many runnable processes

CHECK:

```
top, mpstat -P ALL 1, ps -eo pid,ppid,cmd,%cpu --sort=-%cpu
```



MEMORY BOUND

- Low available memory
- High used memory
- Swap used (si/so > 0 in vmstat)
- Processes may be killed (OOM)

CHECK:

```
free -h, vmstat 1, dmesg | grep -i oom
```



DISK / I/O BOUND

- High %util in iostat
- High await / svctm
- High iowait in top/mpstat
- Load average high, CPU idle high

CHECK:

```
iostat -xz 1, top, dmesg | grep -i error
```

COMMAND QUICK REFERENCE

COMMAND	PURPOSE
free -h	Show memory and swap usage
vmstat 1	Report virtual memory statistics
iostat -xz 1	Show CPU and disk I/O statistics
mpstat -P ALL 1	Show per-CPU statistics
top	Real-time process and resource usage
uptime	Show uptime and load averages

ADD 1 <interval_seconds> FOR REPEATED REFRESH

KEY METRICS EXPLAINED

METRIC	WHAT IT MEANS
Load Average	Average number of running + waiting processes
%CPU (usr, sys, id)	User, system, idle CPU time
Available Memory	Memory available for new applications
Swap Used	If swap fills up, system will start to slow
%iowait	CPU waiting for disk I/O (should be low)
%util (Disk)	Disk busy percentage (over 80% is high)
await	Average time for I/O request (ms)



TIPS

- Correlate high load with CPU, memory, and disk metrics.
- Always check dmesg for hardware or OOM messages.
- Use longer intervals (5-10s) for stable metrics.



COMMON PERFORMANCE BOTTLENECKS

- High CPU usage from a runaway process.
- Not enough memory leading to swap usage.
- Slow disks (high %util or await).
- Too many small I/O operations.

VERIQTA SOCIAL

VERIQTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ip • ping • ss • curl • wget • dig • nslookup • traceroute • nc



ESSENTIAL NETWORKING COMMANDS TO DIAGNOSE, TEST, AND TROUBLESHOOT NETWORK CONNECTIVITY.

1. NETWORK INTERFACES



ip a

Show all network interfaces and IP addresses.

```
$ ip a
1: lo: <LOOPBACK...>
   inet 127.0.0.1/8
2: eth0: <BROADCAST...>
   inet 192.168.1.10/24
```

2. IP ADDRESSES



ip -4 addr show

Show IPv4 addresses only.

```
$ ip -4 addr show
eth0      inet 192.168.1.10/24
eth1      inet 10.0.0.5/24
```

3. ROUTING TABLE



ip route show

Show IP routing table and default gateway.

```
$ ip route show
default via 192.168.1.1 dev eth0
10.0.0.0/24 dev eth1 proto kernel
192.168.1.0/24 dev eth0 proto kernel
```

4. DNS RESOLUTION (DIG)



dig example.com

Query DNS records for a domain.

```
$ dig example.com
;; ANSWER SECTION:
example.com. 300 IN A 93.184.216.34
```

5. DNS RESOLUTION (NSLOOKUP)



nslookup example.com

Resolve domain to IP address.

```
$ nslookup example.com
Server:      8.8.8.8
Address:     8.8.8.8
Name:        example.com
Address:     93.184.216.34
```

6. CONNECTIVITY TEST (PING)



ping -c 4 8.8.8.8

Test connectivity to a host.

```
$ ping -c 4 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1
64 bytes from 8.8.8.8: icmp_seq=2
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received,
0% packet loss, time 3003ms
```

7. TRACEROUTE



traceroute example.com

Trace the network path to a host.

```
$ traceroute example.com
1  192.168.1.1    1.123 ms
2  10.0.0.1       3.456 ms
3  93.184.216.34 12.789 ms
```

8. CHECK OPEN PORTS (NC)



nc -zv 93.184.216.34 80

Check if a port is open on a remote host.

```
$ nc -zv 93.184.216.34 80
Connection to 93.184.216.34 80
port [tcp/http] succeeded!
```

9. LISTENING PORTS (SS)



ss -tulnp

Show listening TCP and UDP ports with processes.

```
$ ss -tulnp
Netid State Recv-Q Send-Q Local Address:Port
tcp    LISTEN 0      128  0.0.0.0:22    users:("sshd",pid=1234)
tcp    LISTEN 0      128  0.0.0.0:80    users:("nginx",pid=2345)
udp    UNCONN 0      0    0.0.0.0:53    users:("systemd-resolve",pid=789)
```

10. HTTP TEST (CURL)

>_ curl

curl -I https://example.com

Send HTTP HEAD request and show response headers.

```
$ curl -I https://example.com
HTTP/2 200
date: Sat, 04 May 2024 12:00:00 GMT
server: nginx
content-type: text/html
content-length: 1256
```

11. DOWNLOAD FILE (WGET)



wget -O file.html https://example.com

Download a file from the web.

```
$ wget -O file.html https://example.com
--2024-05-04 12:00:00-- https://example.com/
Resolving example.com (example.com)... 93.184.216.34
Connecting to example.com (example.com)[93.184.216.34]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1256 (1.2K) [text/html]
Saving to: 'file.html'
file.html 100%[=====] 1.2K --.-KB/s in 0s
2024-05-04 12:00:01 (20.4 MB/s) - 'file.html' saved [1256/1256]
```

COMMON NETWORK INTERFACE COMMANDS (IP)

ip link show	Show all network interfaces
ip addr show	Show IP addresses
ip addr add 10.0.0.10/24 dev eth0	Add IP address to interface
ip link set eth0 up	Bring interface up
ip link set eth0 down	Bring interface down
ip route show	Show routing table
ip route add 10.0.0.0/24 via 192.168.1.1	Add static route

QUICK CONNECTIVITY CHECK FLOW



1. Interface Up?

ip link



2. IP Address Configured?

ip addr



3. Can You Ping?

ping



4. DNS Working?

dig / nslookup



5. Route Correct?

ip route



6. Port Open?

ss / nc



7. App / HTTP Responding?

curl / wget

DNS QUICK COMMANDS

dig A example.com	Get A record
dig MX example.com	Get MX record
dig +short example.com	Short answer
dig @1.1.1.1 example.com	Query specific DNS server

TROUBLESHOOTING TIPS

- No connectivity? Check interface, IP, route, firewall.
- DNS issues? Test with dig and nslookup.
- Port closed? Check service status and firewall.
- Slow network? Use traceroute and check MTU.
- Always test from source to destination.

COMMON PORTS REFERENCE

22	SSH	5432	PostgreSQL
53	DNS	6379	Redis
80	HTTP	8080	HTTP-Alt
443	HTTPS	9090	Prometheus
3306	MySQL		

15

SSH AND REMOTE ADMINISTRATION

Page 15 of 20

ssh • scp • sftp • rsync

**SECURELY CONNECT TO REMOTE SYSTEMS, TRANSFER FILES, AND MANAGE SERVERS OVER ENCRYPTED SSH.**

1. SSH CONNECTIONS

**ssh user@host**

Connect to a remote host securely.

```
ssh user@192.168.1.10
ssh -p 2222 user@host
```

2. SSH KEYS OVERVIEW



- More secure than passwords
- Key pair: public & private
- Private key stays on your host
- Public key on remote host

3. GENERATE SSH KEY (SSH-KEYGEN)

**ssh-keygen -t ed25519 -C "you@example.com"**

```
ssh-keygen -t ed25519 -C
"you@example.com"
Enter file in which to save
(/home/user/.ssh/id_ed25519):
Enter passphrase (optional):
```

4. COPY SSH KEY TO REMOTE (SSH-COPY-ID)

**ssh-copy-id user@host**

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub
user@192.168.1.10
Now try logging into the machine,
with: "ssh "user@192.168.1.10"
and check to make sure that only
the key(s) you wanted were added.
```

5. SECURE COPY (SCP)

**scp [options] source user@host:destination**

```
scp file.txt user@host:/home/user/
scp -r /local/dir user@host:/remote/dir
scp -P 2222 file.txt user@host:/tmp/
```

6. SFTP (SECURE FILE TRANSFER)

**sftp user@host**

```
sftp> ls
sftp> cd /remote/dir
sftp> put file.txt
sftp> get file.txt
sftp> bye
```

7. REMOTE SYNCHRONIZATION (RSYNC)

**rsync [options] source user@host:destination**

```
rsync -avz /local/dir/ user@host:/
rsync -avz --delete /local/dir/
user@host:/remote/dir/
rsync -avz -e "ssh -p 2222"
/local/dir/ user@host:/data/
```

8. SSH CONFIG FILE (~/.ssh/config)



- Simplify connections
- Store usernames, ports, and keys

```
Host myserver
HostName 192.168.1.10
User ubuntu
Port 2222
IdentityFile ~/.ssh/id_ed25519
ServerAliveInterval 60
```

9. FILE PERMISSIONS FOR SSH KEYS



- Private key must be protected
- Incorrect permissions cause errors

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub
```

10. COMMON SSH FAILURES & FIXES



PROBLEM	CAUSE & FIX
Permission denied (publickey)	Key not added or wrong permissions. Fix: Check ~/.ssh/permissions and authorized_keys.
Connection timed out	Host down, firewall, or wrong IP/port. Fix: Check network, port, and firewall rules.
Host key verification failed	Host key changed or unknown. Fix: ssh-keygen -R host or accept new key.
Too many authentication failures	Wrong user/key or brute force protection. Fix: Verify user and key, wait or allow in firewall.
Connection reset by peer	SSH service down or network device reset. Fix: Check SSH service and network stability.

11. SSH COMMAND QUICK REFERENCE

COMMAND	PURPOSE
ssh user@host	Connect to remote host
ssh -p 2222 user@host	Connect using custom port
ssh -i key.pem user@host	Connect using specific key
ssh -v user@host	Verbose mode for debugging
scp file user@host:/path/	Copy file to remote host
scp user@host:/path/file .	Copy file from remote host
sftp user@host	Interactive secure file transfer
rsync -avz src/ user@host:/dest/	Sync files/directories
ssh-keygen -t ed25519	Generate SSH key pair
ssh-copy-id user@host	Copy public key to remote host
ssh-keyscan host	Scan host keys
ssh -L 8080:localhost:80 user@host	Local port forwarding
ssh -N -f user@host	Run SSH in background
ssh -T git@github.com	Test SSH connection (no shell)

12. SSH BEST PRACTICES

- ✓ Use SSH keys, not passwords.
- ✓ Use strong passphrases for private keys.
- ✓ Keep private keys secure and backed up.
- ✓ Disable root login on servers.
- ✓ Use non-standard SSH ports if appropriate.
- ✓ Keep SSH software updated.
- ✓ Use Fail2Ban or firewall to prevent brute force.
- ✓ Review ~/.ssh/authorized_keys regularly.



SECURE REMOTE ACCESS

SSH is the backbone of remote administration. Secure it, monitor it, and automate it!

13. COMMON PORTS

PORT	SERVICE	DESCRIPTION
22	SSH	Secure Shell
2222	SSH (Alt)	Alternative SSH Port
443	HTTPS	Secure Web Traffic
80	HTTP	Web Traffic
21	FTP	File Transfer Protocol
990	FTPS	Secure FTP

VERIQTA SOCIAL

VERIQTA



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

16

ARCHIVES, COMPRESSION,
AND FILE TRANSFER

Page 16 of 20

tar • gzip • gunzip • xz • zip • unzip • rsync

CREATE ARCHIVES, COMPRESS FILES, EXTRACT CONTENT,
AND TRANSFER FILES AND DIRECTORIES EFFICIENTLY.

1. TAR ARCHIVES

`tar -cvf archive.tar file1 file2 dir/`

- c = create
- v = verbose
- f = file
- x = extract

Common Options:

- cvf Create archive
- xvf Extract archive
- tf List archive contents
- czvf Create gzip compressed tar
- xZvf Extract xz compressed tar

```
$ tar -cvf backup.tar /etc/home/user
$ tar -xvf backup.tar
$ tar -tf backup.tar
```

2. GZIP COMPRESSION

`gzip file.txt`

- Compress a file (creates file.txt.gz)
- Original file is removed by default

COMMON COMMANDS:

```
gzip file.txt
gzip -c file.txt > file.txt.gz
gunzip file.txt.gz
zcat file.txt.gz (view without extracting)
```

```
$ gzip file.txt
$ ls
file.txt.gz
$ gunzip file.txt.gz
```

3. BZIP2 COMPRESSION

`bzip2 file.txt`

- Compress a file (creates file.txt.bz2)
- Better compression than gzip, but slower

COMMON COMMANDS:

```
bzip2 file.txt
bzip2 -c file.txt > file.txt.bz2
bunzip2 file.txt.bz2
bzcat file.txt.bz2 (view without extracting)
```

```
$ bzip2 file.txt
$ ls
file.txt.bz2
$ bunzip2 file.txt.bz2
```

4. XZ COMPRESSION

`xz file.txt`

- Compress a file (creates file.txt.xz)
- Highest compression ratio, but slowest

COMMON COMMANDS:

```
xz file.txt
xz -c file.txt > file.txt.xz
unxz file.txt.xz
xzcat file.txt.xz (view without extracting)
```

```
$ xz file.txt
$ ls
file.txt.xz
$ unxz file.txt.xz
```

5. ZIP COMPRESSION

`zip archive.zip file1 file2 dir/`

- Create a zip archive
- Great for cross-platform usage

COMMON COMMANDS:

```
zip archive.zip file1 file2 dir/
zip -r archive.zip dir/
zip -e archive.zip file.txt
zip -e secret.zip secret.txt
```

```
$ zip -r project.zip project/
$ zip -e secret.zip secret.txt
```

6. UNZIP (EXTRACT ZIP)

`unzip archive.zip`

- Extract files from zip archive
- Preserves directory structure

COMMON COMMANDS:

```
unzip archive.zip
unzip -l archive.zip (list contents)
unzip -d /path/to/dir archive.zip
(extract to specific directory)
```

```
$ unzip archive.zip
$ unzip -l archive.zip
$ unzip -d /path/to/dir archive.zip
```

7. EXTRACT TAR ARCHIVES

`tar -xvf archive.tar`

- Extract tar archive
- Use -z for .tar.gz or -J for .tar.xz

EXAMPLES:

```
tar -xvzf archive.tar.gz
tar -xjvf archive.tar.bz2
tar -xJvf archive.tar.xz
```

```
$ tar -xvzf backup.tar.gz
$ tar -xjvf backup.tar.bz2
$ tar -xJvf backup.tar.xz
```

8. PRESERVE PERMISSIONS

Always preserve permissions
during archiving and transfer.

- tar preserves permissions by default
- Use -p with other tools when available
- Avoid extracting as different user unless necessary

```
$ tar -cvpf archive.tar dir/
$ tar -xvpf archive.tar
(p = preserve permissions)
```

9. TRANSFER DIRECTORIES (RSYNC)

`rsync -avz source/ user@host:/destination/`

- Fast, efficient, and resumable
- Preserves timestamps, permissions, links

COMMON RSYNC OPTIONS:

- a Archive mode (preserve everything)
- v Verbose
- z Compress during transfer
- h Human readable
- delete Delete files on destination that don't exist on source

```
$ rsync -avz /home/user/docs/ user@server:/backup/docs/
$ rsync -avz --delete /var/www/ user@server:/var/www/
```

10. OTHER FILE TRANSFER TOOLS

`scp file.txt user@host:/path/`Secure copy file
to remote host.`sftp user@host`Interactive secure
file transfer.

EXAMPLES:

```
scp file.txt user@host:/home/user/
scp -r dir/ user@host:/path/
sftp user@host
```

11. QUICK REFERENCE TABLE

TASK	COMMAND EXAMPLE
Create tar archive	<code>tar -cvf archive.tar dir/</code>
Extract tar archive	<code>tar -xvf archive.tar</code>
Create tar.gz archive	<code>tar -czvf archive.tar.gz dir/</code>
Extract tar.gz archive	<code>tar -xvzf archive.tar.gz</code>
Create tar.bz2 archive	<code>tar -cjvf archive.tar.bz2 dir/</code>
Extract tar.bz2 archive	<code>tar -xjvf archive.tar.bz2</code>
Create tar.xz archive	<code>tar -cJvf archive.tar.xz dir/</code>
Extract tar.xz archive	<code>tar -xJvf archive.tar.xz</code>
Compress with gzip	<code>gzip file.txt</code>
Decompress gzip	<code>gunzip file.txt.gz</code>
Compress with bzip2	<code>bzip2 file.txt</code>
Decompress bzip2	<code>bunzip2 file.txt.bz2</code>
Compress with xz	<code>xz file.txt</code>
Decompress xz	<code>unxz file.txt.xz</code>
Create zip archive	<code>zip -r archive.zip dir/</code>
Extract zip archive	<code>unzip archive.zip</code>
Sync directories	<code>rsync -avz source/ dest/</code>
Secure copy file	<code>scp file.txt user@host:/path/</code>



BEST PRACTICES

- Use tar for backups (best for Linux/Unix).
- Use gzip for speed, bzip2/xz for better compression.
- Always test archives after creation.
- Use rsync for large data or frequent sync.
- Keep permissions and ownership consistent.



COMMON FILE EXTENSIONS

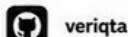
.tar	TAR archive	.gz	Gzip compressed file
.tar.gz	Gzip compressed TAR	.bz2	Bzip2 compressed file
.tar.bz2	Bzip2 compressed TAR	.xz	XZ compressed file
.tar.xz	XZ compressed TAR	.zip	Zip archive

VERIQTA SOCIAL

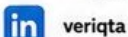
VERIQTA



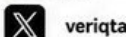
veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

17

LOGS AND JOURNALCTL

Page 17 of 20

journalctl • tail • grep • dmesg



MONITOR, SEARCH, AND ANALYZE SYSTEM LOGS TO TROUBLESHOOT ISSUES AND INVESTIGATE INCIDENTS.

1. LOG FILES LOCATION (/VAR/LOG)



- Traditional log files stored under /var/log.
- Plain text logs, rotated by logrotate.

```
$ ls -lah /var/log
$ ls -lh /var/log/*.log
$ du -sh /var/log
```

2. SYSTEMD JOURNAL



- Binary log store managed by systemd-journald.
- More efficient and reliable than plain text logs.

```
$ journalctl
$ journalctl --help
$ journalctl --disk-usage
```

3. FOLLOW LOGS LIVE (REAL-TIME)



- Stream logs in real-time as they are written.

```
$ journalctl -f
$ tail -f /var/log/syslog
$ tail -f /var/log/messages
```

4. FILTER BY SERVICE



- View logs for a specific service or unit.

```
$ journalctl -u nginx
$ journalctl -u ssh
$ journalctl -u systemd
```

5. FILTER BY TIME



- View logs within a time range.

```
$ journalctl --since "2024-05-04 00:00"
$ journalctl --since "yesterday"
$ journalctl --until "2024-05-04 12:00"
$ journalctl --since "-2h"
```

6. KERNEL MESSAGES (DMESG)



- View kernel ring buffer messages.
- Useful for hardware, driver, and boot issues.

```
$ dmesg
$ dmesg -T
$ dmesg | less
```

7. AUTHENTICATION LOGS



- Monitor login attempts, sudo, SSH, failures, and user actions.

```
$ tail -f /var/log/auth.log
$ grep "Failed password" /var/log/auth.log
$ last -n 20
$ lastb -n 20
```

8. SEARCH ERRORS IN LOGS



- Quickly find errors, failures, or critical issues.

```
$ journalctl -p err
$ journalctl -p crit
$ grep -i error /var/log/syslog
$ grep -i fail /var/log/syslog
```



9. JOURNALCTL COMMON COMMANDS

COMMAND	DESCRIPTION
journalctl	Show all logs
journalctl -f	Follow logs live
journalctl -n 50	Show last 50 log entries
journalctl -u <service>	Logs for a specific service
journalctl --since "time"	Logs since given time
journalctl --until "time"	Logs until given time
journalctl -p err	Show error and above
journalctl -b	Logs from current boot
journalctl -b -1	Logs from previous boot
journalctl --disk-usage	Show journal disk usage



10. TAIL AND GREP COMBINATIONS

```
$ tail -f /var/log/syslog
Follow syslog live

$ tail -f /var/log/syslog | grep nginx
Follow and filter nginx logs

$ grep -i error /var/log/syslog
Search for errors (case-insensitive)

$ grep -i fail /var/log/auth.log
Search authentication failures
```

11. COMMON LOG FILES REFERENCE

FILE	PURPOSE
/var/log/syslog	General system logs (Debian/Ubuntu)
/var/log/messages	General system logs (RHEL/CentOS)
/var/log/auth.log	Authentication logs (Debian/Ubuntu)
/var/log/secure	Authentication logs (RHEL/CentOS)
/var/log/kern.log	Kernel logs
/var/log/dmesg	Kernel ring buffer (alt location)
/var/log/daemon.log	System daemons
/var/log/nginx/	Nginx access/error logs
/var/log/apache2/	Apache access/error logs
/var/log/boot.log	System boot logs



12. PRODUCTION LOG INVESTIGATION WORKFLOW

1. IDENTIFY THE ISSUE



Understand the alert, error message, or user report.

2. CHECK SYSTEM OVERVIEW



Check system status, uptime, load, and resources.

3. CHECK RELEVANT LOG SOURCES



Identify which logs (journalctl, files) may contain details.

4. FILTER BY TIME & SERVICE



Use time range and service filters to narrow results.

5. SEARCH FOR ERRORS



Search for errors, warnings, and failed events.

6. CORRELATE & ANALYZE



Correlate logs, identify root cause, and impacted scope.

7. TAKE ACTION & MONITOR



Fix the issue, monitor logs, and verify recovery.

13. QUICK REFERENCE COMMANDS



```
journalctl -f          Follow systemd journal live
journalctl -u nginx -f Follow nginx logs (journal)
journalctl --since "-1h" Logs from last 1 hour
journalctl -p err..alert Show error and critical logs
journalctl -k          Show kernel messages
dmesg -T | less        Kernel messages with timestamps
tail -f /var/log/syslog Follow syslog live
grep -i error /var/log/syslog Search errors in syslog
```

14. BEST PRACTICES



- Centralize logs with rsyslog or systemd-journal-remote.
- Rotate logs to prevent disk space exhaustion (logrotate).
- Monitor authentication logs for suspicious activity.
- Set alerts for critical errors and repeated failures.
- Use timestamps and service filters for accurate analysis.
- Maintain log retention policies based on compliance needs.

18

BASH, ENVIRONMENT, AND SHELL PRODUCTIVITY

Page 18 of 20

export • env • history • alias • echo • > • >> • 2> • 2> &1 • | • \$?



WORK SMARTER IN THE SHELL. AUTOMATE, CUSTOMIZE, AND CONTROL INPUT AND OUTPUT EFFECTIVELY.

1. ENVIRONMENT VARIABLES



- System-wide and user-defined variables
- Available to current session and child processes

```
$ env          # Show all environment variables
$ printenv     # Same as env
$ echo $HOME
$ echo $SHELL
$ echo $USER
$ echo $PWD
```

2. PATH VARIABLE



- Defines directories the shell searches for commands
- Add custom bin directories to PATH

```
$ echo $PATH
$ export PATH="$PATH:/usr/local/bin"
$ export PATH="/opt/tools/bin:$PATH"
$ which python3
$ command -v docker
```

3. SHELL VARIABLES



- Create and use variables in your shell
- Not available to child shells unless exported

```
$ NAME="veriqta"
$ echo $NAME
$ COUNT=10
$ echo $COUNT
$ readonly SITE="veriqta.com"
$ unset COUNT      # Remove variable
```

4. COMMAND HISTORY



- View and manage previously used commands
- Improve productivity with history search

```
$ history      # Show command history
$ history 10   # Show last 10 commands
$ !n          # Re-run command number n
$ !!         # Re-run last command
$ !string     # Re-run last command starting with string
$ history -c   # Clear history
```

5. ALIASES



- Create shortcuts for long or frequently used commands
- Aliases improve speed and consistency

```
$ alias ll='ls -alF'
$ alias la='ls -A'
$ alias gs='git status'
$ alias k='kubectl'
$ alias      # List all aliases
$ unalias ll # Remove alias
$ alias ll   # Show specific alias
```

6. ECHO COMMAND



- Print text and variables to the terminal
- Useful in scripts and debugging

```
$ echo "Hello, Linux!"
$ echo $USER
$ echo "Home: $HOME"
$ echo -e "Line1\nLine2"
$ echo -n "No newline" # No trailing newline
```

7. INPUT / OUTPUT REDIRECTION



- Redirect command output to files
- Overwrite or append
- Redirect errors

SYMBOL	DESCRIPTION	EXAMPLE
>	Redirect stdout (overwrite)	ls -l > out.txt
>>	Redirect stdout (append)	echo "hi" >> out.txt
2>	Redirect stderr (overwrite)	ls /bad 2> error.log
2>>	Redirect stderr (append)	cmd 2>> error.log
&>	Redirect stdout & stderr	cmd &> all.log
2>&1	Redirect stderr to stdout	cmd > out.log 2>&1

8. PIPES



- Send output of one command as input to another
- Combine commands for powerful workflows

```
$ ps aux | grep nginx
$ df -h | sort -hr
$ cat file.txt | grep "error" | wc -l
$ ls -l /var/log | awk '{print $9}'
$ top -b -n 1 | head -20
```

9. EXIT CODES



- Each command returns a status code
- 0 = success, non-zero = failure

```
$ command
$ echo $?      # Show last command exit code
$ ls /no/such/dir
$ echo $?      # Example: 2 (No such file)
$ false
$ echo $?      # 1 (General error)
$ true
$ echo $?      # 0 (Success)
```

10. COMMAND SUBSTITUTION



- Use the output of a command inside another
- Improves scripting and automation

```
$ USER_COUNT=$(who | wc -l)
$ echo "Logged in users: $USER_COUNT"
$ CURRENT_DATE=$(date +%F)
$ echo "Today is $CURRENT_DATE"
$ FILES=$(ls /var/log | wc -l)
$ echo "Log files: $FILES"
```

11. USEFUL BUILT-IN COMMANDS

COMMAND	PURPOSE
export	Set export environment variables
env	Display environment variables
history	Show command history
alias	Create or list aliases
echo	Display a line of text
type	Show if command is alias, built-in, or executable
hash	Show command locations saved in hash table
pwd	Print working directory
read	Read input from user
shopt	Shell options

12. REDIRECTION & PIPES EXAMPLES

EXAMPLE	DESCRIPTION
ls -l > list.txt	Save output to file (overwrite)
ls -l >> list.txt	Append output to file
find / -name error.log 2> errors.txt	Save errors to file
find / -name error.log &> all.txt	Save stdout and stderr
cmd 2>&1 tee output.log	Redirect & display output
cat file.txt grep ERROR	Filter lines containing ERROR
ps aux grep java wc -l	Count Java processes
dmesg less	View kernel messages

13. BEST PRACTICES



- Use meaningful aliases and keep them simple.
- Add useful aliases to ~/.bashrc or ~/.bash_aliases.
- Always quote variables: "\$VAR".
- Check exit codes in scripts to handle failures.

14. QUICK REFERENCE

export VAR=value	Set environment variable	> file	Redirect output (overwrite)
env	Show environment variables	>> file	Redirect output (append)
history	Show command history	2> file	Redirect errors
alias name='cmd'	Create alias	2>&1	Redirect errors to output
unalias name	Remove alias		Pipe output to next command
echo "text"	Print text	\$?	Last command exit code

19

LINUX SECURITY AND
PRODUCTION HARDENING

Page 19 of 20

sudo • ss • last • lastb • journalctl • ufw • firewall-cmd

SECURE YOUR SYSTEM, REDUCE RISK, AND DETECT THREATS
BEFORE THEY BECOME INCIDENTS.

1. USER PRIVILEGE CHECKS



- Check current user and groups
- Verify sudo privileges

```
$ whoami
$ id
$ groups
$ sudo -l          # List sudo
                    # privileges
```

2. FILE PERMISSION AUDITS



- Find world-writable files
- Identify risky permissions

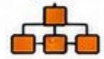
```
$ find / -xdev -type f -perm -0002
-ls 2>/dev/null
$ find / -xdev -type d -perm -0002
-ls 2>/dev/null
$ ls -l /etc/passwd /etc/shadow
```

3. SSH HARDENING CHECKS



- Review SSH configuration
- Ensure strong authentication

```
$ grep -E '^(PermitRootLogin|
PasswordAuthentication|
ChallengeResponseAuthentication)'
/etc/ssh/sshd_config
$ systemctl status sshd
$ journalctl -u sshd --no-pager -n 20
```

4. OPEN-PORT REVIEW
(SS)

- List listening ports
- Review exposure

```
$ ss -tln
$ ss -tulpn          # Show
                    # processes
$ ss -tunlp | grep LISTEN
```

5. RUNNING SERVICES



- List active services
- Check service status

```
$ systemctl list-units --type=service
--state=running --no-pager
$ systemctl status <service>
$ systemctl is-enabled <service>
```

6. FIREWALL BASICS



- Check firewall status
- Manage firewall rules

UFW (Debian/Ubuntu)

```
$ sudo ufw status verbose
$ sudo ufw enable
$ sudo ufw allow 22/tcp
$ sudo ufw deny 23/tcp
```

FIREWALL-CMD

```
$ sudo firewall-cmd
--state
$ sudo firewall-cmd
--list-all
$ sudo firewall-cmd
--add-service=ssh
--permanent
$ sudo firewall-cmd
--reload
```

7. FAILED LOGIN INVESTIGATION



- Review failed login attempts
- Detect brute-force activity

LASTB

```
$ lastb
$ lastb -n 10
$ lastb -F
/var/log/btmp
```

JOURNALCTL

```
$ journalctl -u sshd
--since '1 hour ago'
| grep 'Failed'
$ journalctl -p err
--since '1 day ago'
| grep -i 'fail'
```

8. SUSPICIOUS PROCESSES



- Identify unusual processes
- Check processes by user

```
$ ps aux --sort=-%cpu | head -n 10
$ ps aux --sort=-%mem | head -n 10
$ top
$ pstree -p
$ lsof -i -P -n
$ lsof -u <user>
```

9. USER SESSION REVIEW (LAST)



- View logged-in users
- Check login history

```
$ last
$ last -n 10
$ last -F /var/log/wtmp
$ w
$ who
```

10. KERNEL & SYSTEM MESSAGES



- Review kernel ring buffer
- Detect hardware & kernel issues

```
$ dmesg | less
$ dmesg -T | tail -n 50
$ journalctl -k --no-pager -n 50
```

11. SUDO SECURITY CHECK



- Review sudo configuration
- Ensure least privilege

```
$ sudo visudo
$ sudo -l
$ grep -r "NOPASSWD" /etc/sudoers*
2>/dev/null
```

12. CRITICAL FILES & DIRECTORIES



- Verify ownership and permissions
- Protect sensitive files

```
$ ls -ld /etc/shadow /etc/passwd
$ ls -l /etc/sudoers /etc/ssh/sshd_config
$ stat /etc/passwd
```

13. PRODUCTION SECURITY CHECKLIST

CHECK ITEM	WHY IT MATTERS	STATUS
✓ 1. Keep system updated	Fix security vulnerabilities	[]
✓ 2. Use strong passwords	Prevent brute-force attacks	[]
✓ 3. Disable root SSH login	Reduce privilege abuse risk	[]
✓ 4. Use SSH keys, not passwords	Stronger authentication	[]
✓ 5. Restrict SSH port (optional)	Reduce automated scans	[]
✓ 6. Enable firewall	Limit unauthorized access	[]
✓ 7. Only allow necessary ports	Minimize attack surface	[]
✓ 8. Disable unused services	Reduce exposure	[]
✓ 9. Monitor failed login attempts	Detect brute-force attacks	[]
✓ 10. Review user privileges	Ensure least privilege	[]
✓ 11. Check file permissions	Prevent data tampering	[]
✓ 12. Monitor logs regularly	Detect suspicious activity	[]
✓ 13. Backup important data	Ensure recovery capability	[]
✓ 14. Enable audit logging (optional)	Track security events	[]

14. COMMON SECURITY COMMAND REFERENCE

COMMAND	PURPOSE
sudo -l	List allowed sudo privileges
ss -tln	List listening ports
last	Show recent user logins
lastb	Show failed login attempts
journalctl -u sshd	View SSH service logs
journalctl -p err	Show error logs
grep -i fail /var/log/auth.log	Search authentication failures
ufw status verbose	Check UFW firewall status
ufw allow 22/tcp	Allow SSH through UFW
firewall-cmd --list-all	List all firewall rules
ps aux	Show running processes
dmesg	Show kernel messages

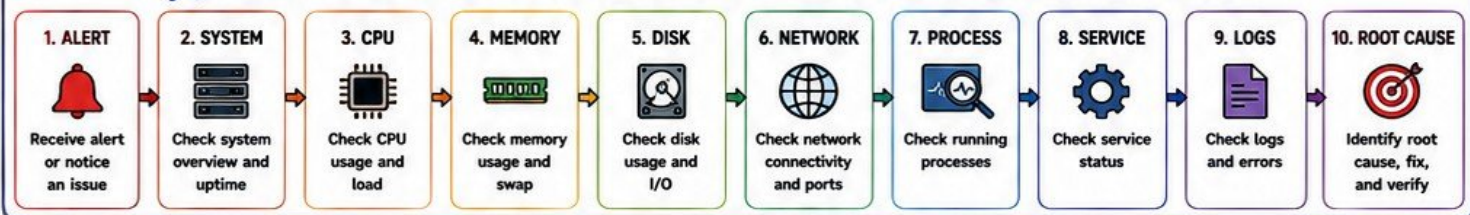
15. PRODUCTION SECURITY WORKFLOW

1. IDENTIFY
THE ISSUE
Detect alert or
suspicious activity2. GATHER
EVIDENCE
Check logs,
users, ports,
and processes3. ANALYZE
DETAILS
Correlate events,
find the root
cause4. CONTAIN
THE RISK
Block IPs, kill
malicious
processes5. REMEDIATE
Fix the issue,
harden system,
apply patches6. MONITOR
& IMPROVE
Continue monitoring
and strengthen
security posture

LINUX PRODUCTION TROUBLESHOOTING CHEAT SHEET



FOLLOW THE FLOW. CHECK EACH LAYER. FIND THE ROOT CAUSE. FIX AND VERIFY.



COMMON PRODUCTION ISSUES AND QUICK COMMANDS

1 SERVER IS SLOW



System feels slow overall.

QUICK CHECKS

```
$ uptime
$ top -b -n 1 | head -20
$ free -h
$ df -h
$ iostat -xz 1 3
$ ss -tln
```

DEEPER CHECKS

```
$ vmstat 1 5
$ pidstat -r -u -d 1 5
$ dmesg | tail -n 50
$ journalctl -p warn...alert --since "1 hour ago"
```

2 CPU IS HIGH



CPU usage is near or at 100%.

QUICK CHECKS

```
$ top -b -n 1 | head -20
$ ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head
$ mpstat -P ALL 1 3
$ pidstat -u 1 5
$ lscpu
```

DEEPER CHECKS

```
$ sar -u 1 5
$ perf top
$ cat /proc/interrupts | column -t
$ systemctl status irqbalance
```

3 MEMORY IS EXHAUSTED



System is using all available memory.

QUICK CHECKS

```
$ free -h
$ top -b -n 1 | head -20
$ swapon --show
$ vmstat 1 5
$ ps aux --sort=-%mem | head
```

DEEPER CHECKS

```
$ cat /proc/meminfo
$ smem -r
$ journalctl -k -p err --since "1 hour ago"
$ dmesg | grep -i oom
```

4 DISK IS FULL



Disk usage is 90% or more.

QUICK CHECKS

```
$ df -h
$ du -xh / | sort -hr | head -20
$ df -i
$ lsblk -f
$ mount | column -t
```

DEEPER CHECKS

```
$ find / -xdev -type f -size +1G -exec ls -lh {} \;
$ journalctl --disk-usage
$ lsof +L1
$ ncdu /
```

5 SERVICE WILL NOT START



Service fails to start or keeps failing.

QUICK CHECKS

```
$ systemctl status <service>
$ systemctl list-units --type=service --all | grep <service>
$ journalctl -u <service> --no-pager -n 50
$ systemctl cat <service>
$ ps aux | grep <service>
```

DEEPER CHECKS

```
$ tail -n 100 /var/log/<service>*.log
$ systemctl reset-failed <service>
$ strace -p <pid>
$ ls -l /etc/<service>/
```

6 PORT IS NOT LISTENING



Expected port is not open or reachable.

QUICK CHECKS

```
$ ss -tln | grep <port>
$ firewall-cmd --list-ports # (RHEL)
$ ufw status verbose # (Ubuntu)
$ nc -zv 127.0.0.1 <port>
$ systemctl status <service>
```

DEEPER CHECKS

```
$ ss -tlnp | grep <port>
$ lsof -i :<port>
$ netstat -tlnp | grep <port>
$ journalctl -u <service> -n 50
```

7 DNS IS FAILING



Hostnames cannot be resolved.

QUICK CHECKS

```
$ dig <domain> | head
$ nslookup <domain>
$ cat /etc/resolv.conf
$ systemd-resolve --status
$ ping <dns-server-ip>
```

DEEPER CHECKS

```
$ dig @<dns-server-ip> <domain>
$ host <domain>
$ journalctl -u systemd-resolved -n 50
$ resolvectl query <domain>
```

8 NETWORK CONNECTION FAILS



Cannot connect to remote host.

QUICK CHECKS

```
$ ping <ip-or-host>
$ traceroute <ip-or-host>
$ ss -tln
$ ip addr
$ ip route
```

DEEPER CHECKS

```
$ mtr -rw <ip-or-host>
$ curl -v http://<ip-or-host>
$ tcpdump -i any host <ip-or-host>
$ ethtool <interface>
```

9 PERMISSION DENIED



Access is denied when running a command or accessing a file.

QUICK CHECKS

```
$ ls -l <file-or-dir>
$ id
$ namei -l <file-or-dir>
$ sudo -l
$ getfacl <file-or-dir>
```

DEEPER CHECKS

```
$ lsattr <file-or-dir>
$ auditctl -l
$ ausearch -m avc -ts recent
$ stat <file-or-dir>
```

10 PROCESS KEEPS CRASHING



Process exits unexpectedly.

QUICK CHECKS

```
$ systemctl status <service>
$ journalctl -u <service> -n 100
$ ps aux | grep <process>
$ dmesg | tail -n 50
$ cat /var/log/messages | tail -n 50
```

DEEPER CHECKS

```
$ coredumpctl list
$ coredumpctl info <PID>
$ systemctl show <service> -p Restart*
$ strace -f -p <PID>
```

11 FILESYSTEM IS READ-ONLY



Filesystem is mounted read-only.

QUICK CHECKS

```
$ mount | grep "ro,"
$ dmesg | tail -n 50
$ df -hT
$ lsblk -o NAME,MOUNTPOINT,RO
$ touch /mnt/testfile
```

DEEPER CHECKS

```
$ fsck -n <device>
$ blkid
$ cat /etc/fstab
$ dmesg | grep -i "error\|readonly"
```

12 SSH CONNECTION FAILS



Unable to login via SSH.

QUICK CHECKS

```
$ systemctl status sshd
$ ss -tln | grep :22
$ tail -n 50 /var/log/auth.log # (Ubuntu)
$ journalctl -u sshd -n 50 # (RHEL)
$ ssh -v user@host
```

DEEPER CHECKS

```
$ cat /etc/ssh/sshd_config
$ lastb | head
$ sudo ausearch -m USER_AUTH -ts recent
$ ufw status # or firewall-cmd --list-all
```



13. RECENT SYSTEM CHANGES

Identify recent changes that may have caused the issue.

```
$ journalctl --since "24 hours ago" --no-pager
$ journalctl -u <service> --since "24 hours ago"
$ ausearch -ts recent
$ cat /var/log/cron
$ rpm -qa --last | head # (RHEL family)
$ dpkg -l | grep "^ii" | sort # (Debian/Ubuntu)
$ stat /etc/cron* -c "%y %n"
$ history | tail -n 50
```



14. PRODUCTION SECURITY CHECKLIST

- ✓ Use strong passwords and SSH keys.
- ✓ Disable root SSH login.
- ✓ Keep system and packages updated.
- ✓ Review and restrict open ports.
- ✓ Use firewall (ufw or firewall-cmd).
- ✓ Monitor failed login attempts.
- ✓ Audit file permissions regularly.
- ✓ Review running services and disable unused ones.
- ✓ Centralize and monitor logs.
- ✓ Enable automatic security updates (where possible).
- ✓ Maintain regular backups.
- ✓ Use least privilege for all processes and users.



15. QUICK REFERENCE COMMAND GROUPS

TASK	COMMANDS
System Overview	uptime, top, free -h, df -h, ip a
CPU	top, mpstat, pidstat, sar -u
Memory	free -h, vmstat, smem
Disk	df -h, du -xh, iostat, lsblk
Network	ip a, ip r, ss -tln, ping, traceroute
Services	systemctl status, systemctl list-units
Logs	journalctl, tail -f, grep
Processes	ps aux, top, htop, pgrep
Security	ss, last, lastb, ufw, firewall-cmd
Authentication	last, lastb, journalctl -u sshd